

Game Engine Concepts

Fachhochschule Oberösterreich

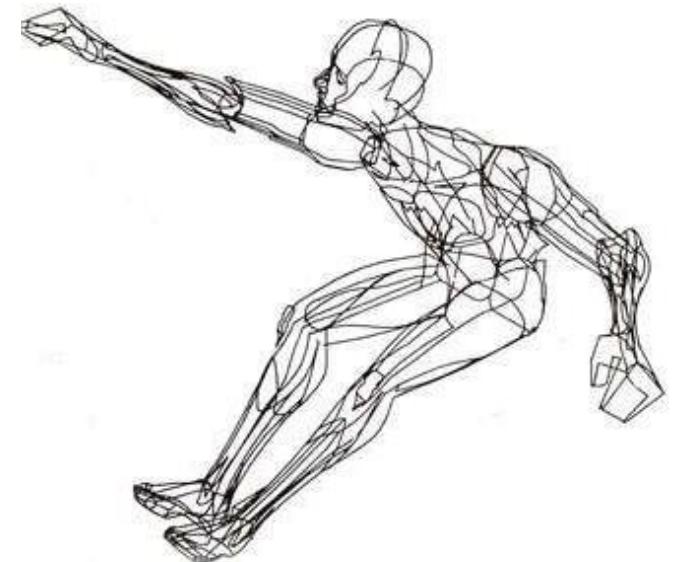
Game Engine Graphics & Rendering

Graphics & Rendering

Introduction

→ Definitions

- Computer graphics are graphics created using computers and, more generally, the representation and manipulation of pictorial data by a computer
- Methods and techniques for converting data to and from graphic displays via a computer
 - Weik 1982
- “Perhaps the best way to define computer graphics is to find out what it is not. It is not a machine. It is not a computer, nor a group of computer programs. It is not the know-how of a graphic designer, programmer, a writer, a motion picture specialist, or a reproduction specialist. Computer graphics is all these – a consciously managed and documented technology directed toward communicating information accurately and descriptively.”
 - William Fetter 1966



Graphics & Rendering

Relevance of Computer Graphics

→ Foundation for graphical digital media and print media

- 2D/3D interfaces, web pages, apps, newspapers, magazines, posters, advertisements, info graphics

→ Foundation for special effects and computer-generated movies

- SFX in blockbusters, animated movies from Pixar and others, but also simple TV like news, game shows, sports

→ Foundation for computer games

- PC, consoles, mobile games, different rendering styles real-time aspects important

→ Foundation for virtual and augmented reality

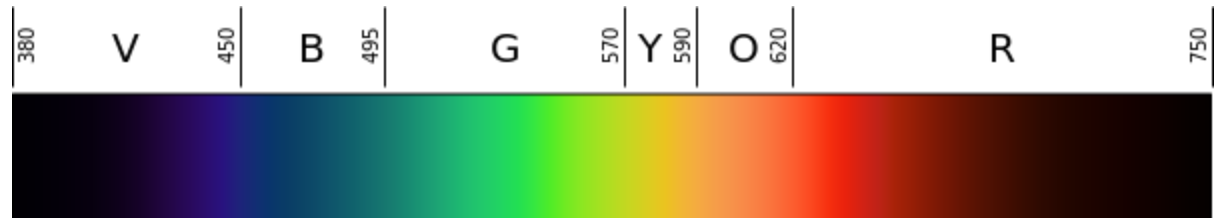
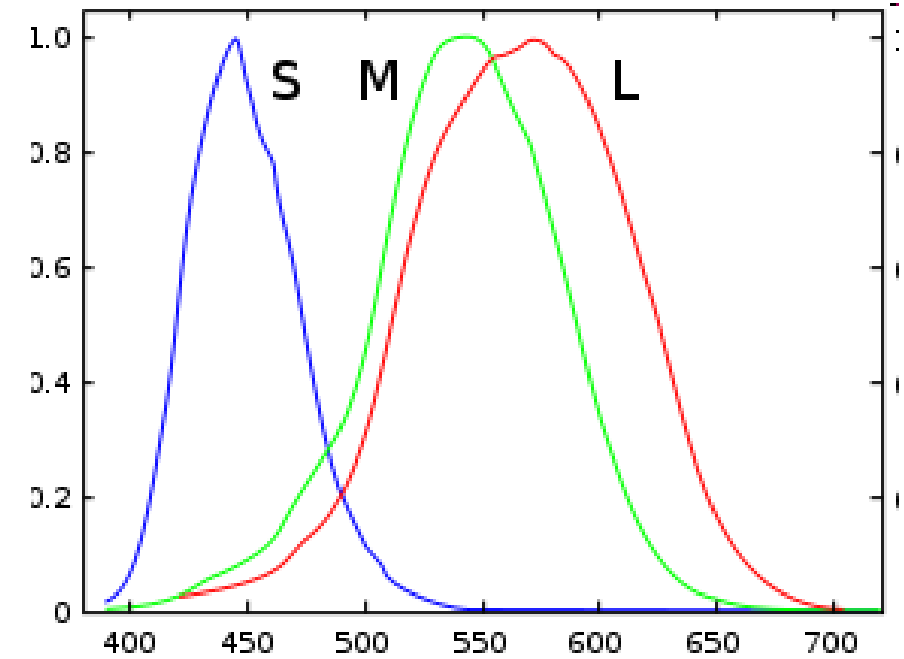
- The additional aspect of stereoscopy plays an important role (covered in AR and VR lecture)



Graphics & Rendering

Visual Perception

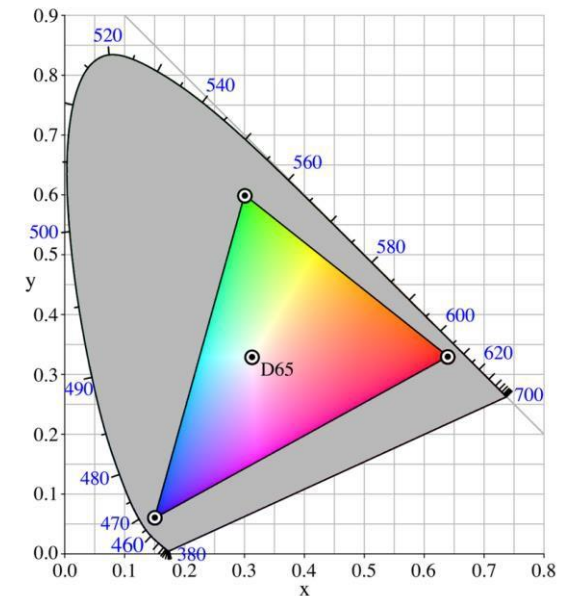
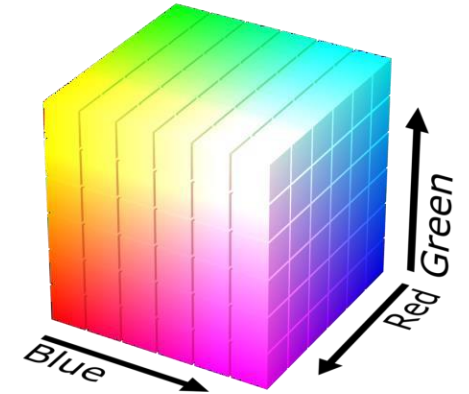
- **Color** is the way how we perceive waves of light
- Perception of an overlap of electromagnetic waves of different wave lengths
- Focus on color perception not visual perception in general
- **Visible electromagnetic spectrum**
 - Ranges from 380 nm (violet) to 740 nm (red)
- Shorter wavelengths (ultraviolet, x-ray) and longer wavelengths (infrared, radio) are not perceived by the human eye
- Not only wavelength but also intensity is relevant for perception



Graphics & Rendering

Red – Green – Blue (RGB)

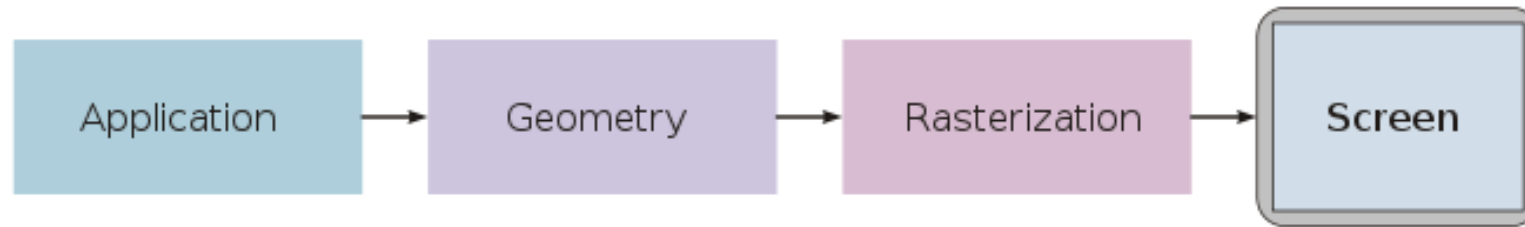
- Color model closely related to display technology (CRT, LCD, plasma, OLED, quantum dots, etc.), thus, the most relevant color model for computer graphics
- Additive color model
- Additional color channel describing the **transparency** (RGBA)
- Description can be mapped on a cube, complementary colors are on the opposite side of the cube
 - Origin (0,0,0) is black
 - (1,1,1) is white
 - Greyscale values lie on the diagonal
- In general, easy to understand but not necessarily intuitive to describe color
- Color gamut only represents subsection of the CIE model



Graphics & Rendering

Graphics Pipeline

- **Rendering pipeline** or graphics pipeline is a model in computer graphics
- It describes the **processing steps from mathematical model to the displayed image**

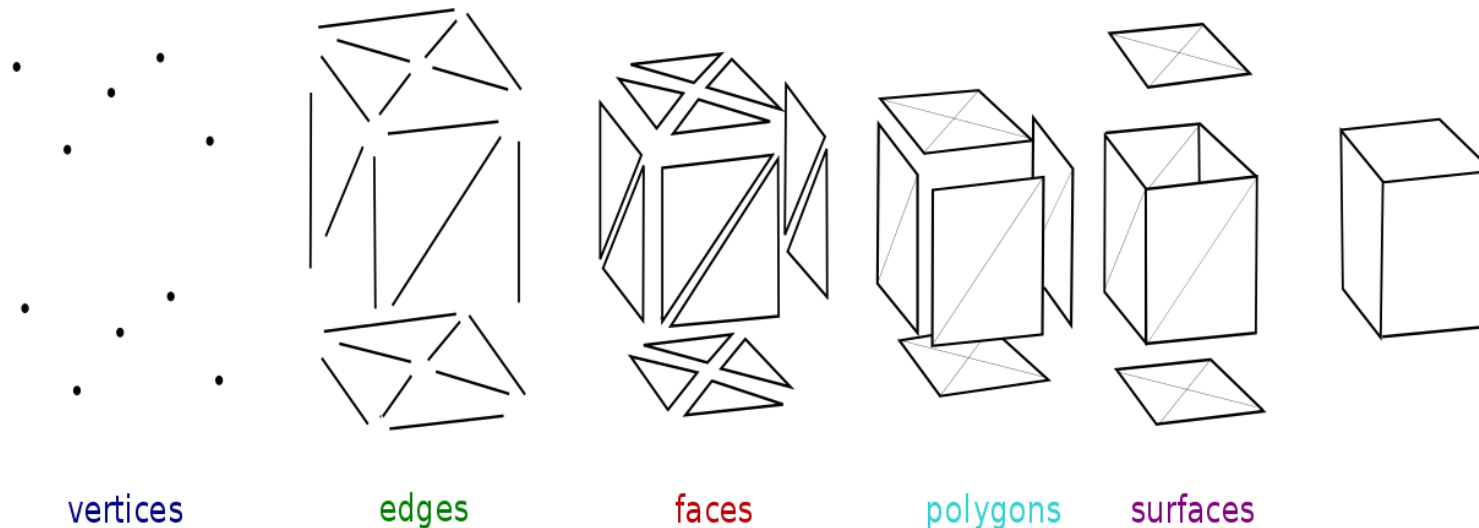


- **Extremely simplified model** (shown above)
 - **Application:** provides the mathematical model and the manipulations applied on it, typically performed on CPU can contain parts like interaction, animation, physics simulation
 - **Geometry:** processing of the geometry, consists of lighting, transformations, projection, clipping etc.
 - **Rasterization:** process of generating a 2D image based on the model defined in the geometry stage
 - **Screen** is the final display medium (2D display in our case)

Graphics & Rendering

Geometric Primitives

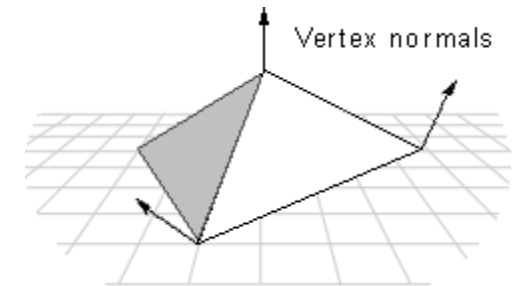
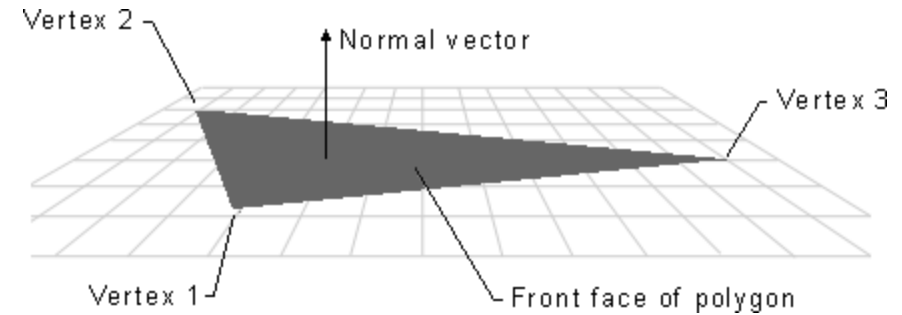
- Typically, two-dimensional and three-dimensional surfaces are formed by **meshes of triangles** (other representations exist e.g. voxels, NURBS)
- Other graphical primitives exist as well
- **Triangle meshes**
 - A triangle (or face) is defined by a set of points (vertex) which is interconnected (edges)
 - Vertices and triangles are characterized by their properties (position, color, normal vectors, texture coordinates, etc.)



Graphics & Rendering

Geometric Primitives

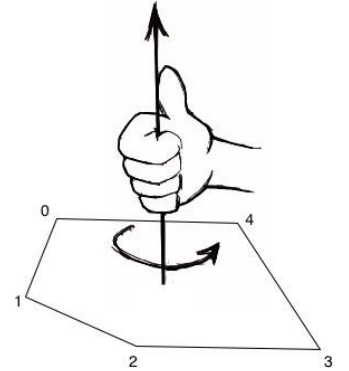
- **Vertex** is a **point** in 3D space (in our case)
- **Triangles** consists of
 - 3 non-collinear vertices interconnected by edges
 - 1 normal vector indicating the front side of the triangle (face) by pointing vertically away from it
 - Additionally normal vectors for the vertices exist
- Knowledge of the front side can be important for **optimization** (more in culling section)
- **Face normal vectors** and vertex normal vectors can be used for determining the **rendered color** of the triangle (more in shading section)



Graphics & Rendering

Geometric Primitives

- Right hand rule is “rule of thumb” to determine the front side (= direction of the normal vector) for a polygon
- Relationship between vertex order and normal vector is a convention not a fixed definition
- Polygons can be **stored in different ways** in memory or on disc (resulting in a variety of file formats e.g. .obj, .3ds, .dae)
- **Examples** would be vertex lists and face lists (description where the individual vertices are located in space combined with a description how they are inter-connected)



Face-Vertex Meshes

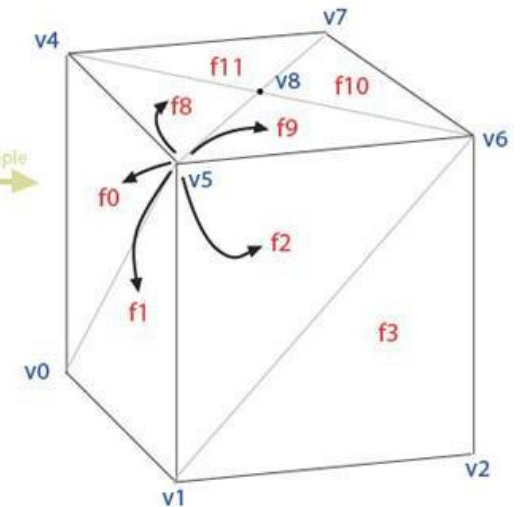
Face List

f0	v0 v4 v5
f1	v0 v5 v1
f2	v1 v5 v6
f3	v1 v6 v2
f4	v2 v6 v7
f5	v2 v7 v3
f6	v3 v7 v4
f7	v3 v4 v0
f8	v8 v5 v4
f9	v8 v6 v5
f10	v8 v7 v6
f11	v8 v4 v7
f12	v9 v5 v4
f13	v9 v6 v5
f14	v9 v7 v6
f15	v9 v4 v7

Vertex List

v0	0,0,0	f0 f1 f12 f15 f7
v1	1,0,0	f2 f3 f13 f12 f1
v2	1,1,0	f4 f5 f14 f13 f3
v3	0,1,0	f6 f7 f15 f14 f5
v4	0,0,1	f6 f7 f0 f8 f11
v5	1,0,1	f0 f1 f2 f9 f8
v6	1,1,1	f2 f3 f4 f10 f9
v7	0,1,1	f4 f5 f6 f11 f10
v8	.5,.5,0	f8 f9 f10 f11
v9	.5,.5,1	f12 f13 f14 f15

example



Graphics & Rendering

Coordinate Systems

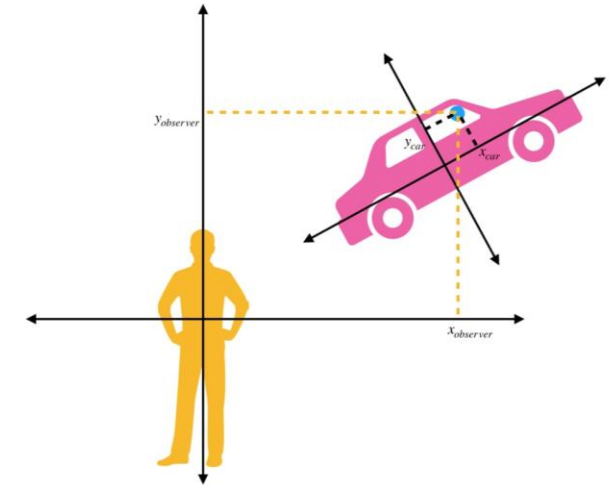
→ In computer graphics two-dimensional and three-dimensional coordinate systems are used

→ 2D

- Screen Coordinate System
- Viewport Coordinate System

→ 3D

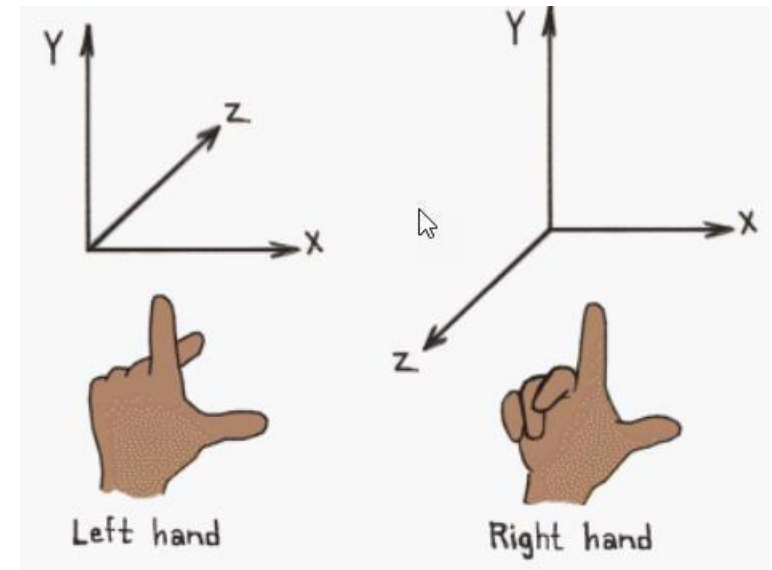
- World Coordinate System
 - Objects are located and placed inside this coordinate system
 - Coordinate system of the virtual world
- Object Coordinate System
 - Local coordinates of each object
 - Each object has own coordinate system
- Hierarchical Coordinate Systems (used in scene graphs)



Graphics & Rendering

Coordinate Systems

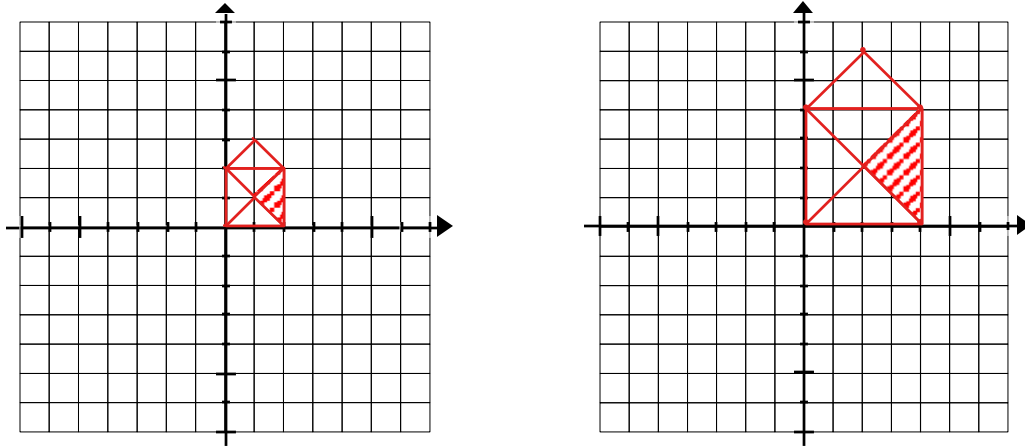
- **Left-handed** vs. **right-handed** coordinate systems
- **Right-handed**
 - X-y plane lies in the screen
 - Z-axis points to the observer
- **Left-handed**
 - X-y plane lies in the screen
 - Z-axis points away from the observer
- Unity uses a left-handed coordinate system, OpenGL uses a right-handed coordinate system
- **Geometrical transformations** are for example translation, scaling, rotation, shearing of objects inside a coordinate system or in-between coordinate systems



Graphics & Rendering

Simple 2D Transformations

→ **Scale** an object by 2



→ We want to get from

$$v_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix} v_2 = \begin{pmatrix} 2 \\ 0 \end{pmatrix} v_3 = \begin{pmatrix} 2 \\ 2 \end{pmatrix} v_4 = \begin{pmatrix} 1 \\ 3 \end{pmatrix} v_5 = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$$

→ to

$$v'_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix} v'_2 = \begin{pmatrix} 4 \\ 0 \end{pmatrix} v'_3 = \begin{pmatrix} 4 \\ 4 \end{pmatrix} v'_4 = \begin{pmatrix} 2 \\ 6 \end{pmatrix} v'_5 = \begin{pmatrix} 0 \\ 4 \end{pmatrix}$$

→ with vector matrix multiplication

→ We need the following **matrix for scaling**

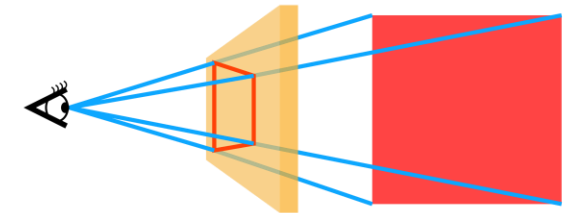
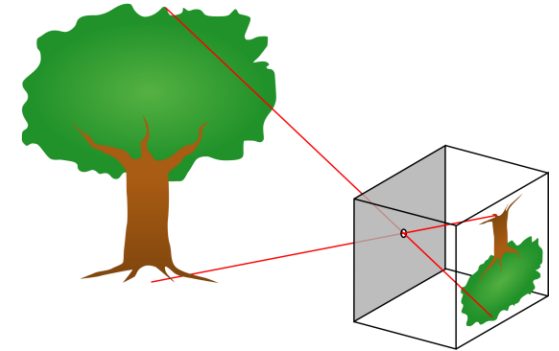
$$scale(s_x, s_y) = \begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}$$

$$\begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 * 0 + 0 * 0 \\ 0 * 0 + 2 * 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$
$$\begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 * 2 + 0 * 0 \\ 0 * 2 + 2 * 0 \end{pmatrix} = \begin{pmatrix} 4 \\ 0 \end{pmatrix}$$
$$\begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 2 \\ 2 \end{pmatrix} = \begin{pmatrix} 2 * 2 + 0 * 2 \\ 0 * 2 + 2 * 2 \end{pmatrix} = \begin{pmatrix} 4 \\ 4 \end{pmatrix}$$
$$\begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 3 \end{pmatrix} = \begin{pmatrix} 2 * 1 + 0 * 3 \\ 0 * 1 + 2 * 3 \end{pmatrix} = \begin{pmatrix} 2 \\ 6 \end{pmatrix}$$
$$\begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 0 \\ 2 \end{pmatrix} = \begin{pmatrix} 2 * 0 + 0 * 2 \\ 0 * 0 + 2 * 2 \end{pmatrix} = \begin{pmatrix} 0 \\ 4 \end{pmatrix}$$

Graphics & Rendering

Projections

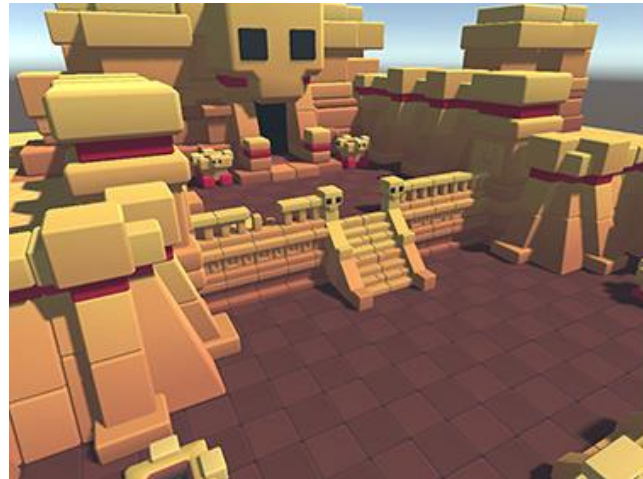
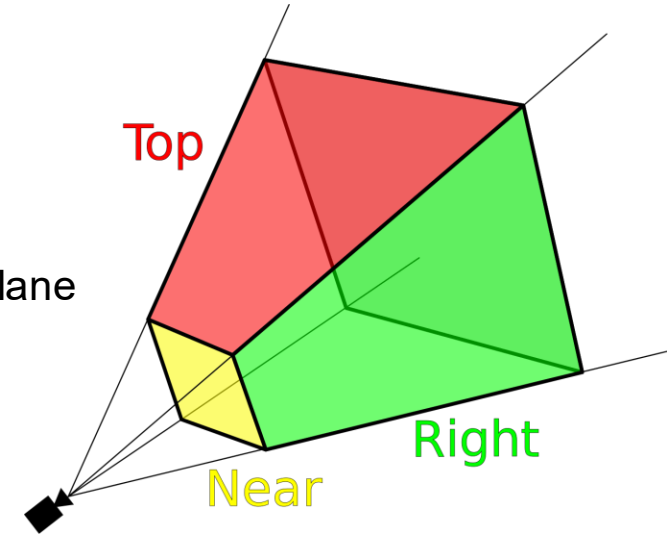
- In **photography**, we usually have the center of projection in-between the object and the image plane
- **Image on film or sensor is recorded upside down**
- In CG perspective projection, the image plane is in front of the camera
- Image in CG is not flipped
- In general, the same mechanisms and rules apply for a perspective projection in graphics as in film
- **Virtual camera** (center of projection) has to be placed in world space as well as the objects in the scene



Graphics & Rendering

Frustum

- The **Frustum** describes a **volume**, which content is projected for rendering
- Areas outside the frustum are not to be rendered
- With **perspective projection** the frustum is a truncated **pyramid**
- Top, bottom, right and left planes restrict the area as well as the near and the far clipping plane
- Alternative naming convention for clipping planes
 - Near Clipping Plane (hither)
 - Far Clipping Plane (yon)
- With **parallel projection** we use a cuboid instead of a frustum with on surface matching the shape of the display plane



Perspective Projection

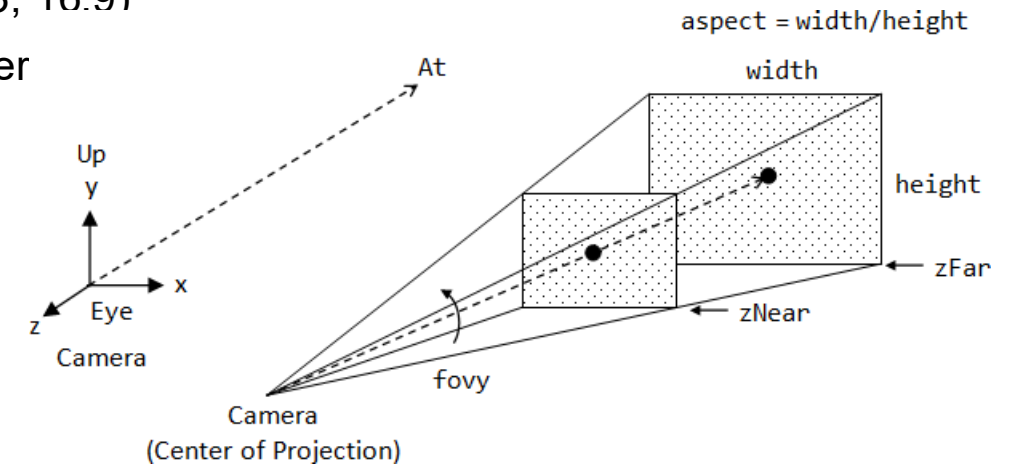


Parallel (Orthographic) Projection

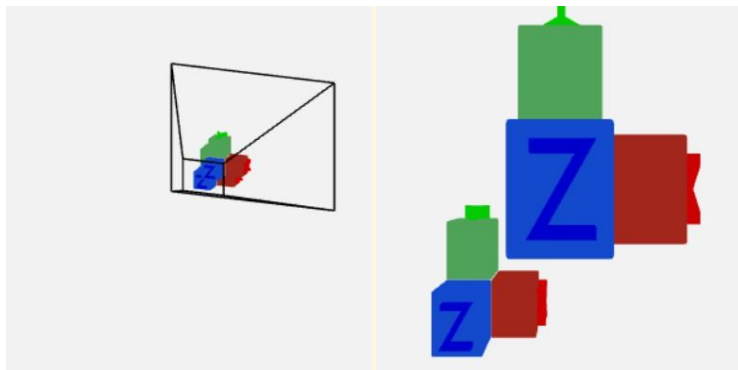
Graphics & Rendering

Perspective Projection

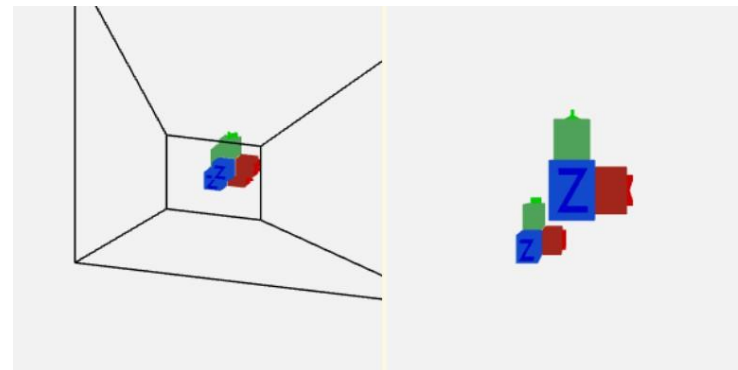
- Other relevant parameters for setting up a frustum for perspective projection
- **Aspect ratio** is the width of the area divided by height of area (e.g. 4:3, 16:9)
- **Field of View (FOV)** is an angle describing the open angle of the camera and/or vertical FOV



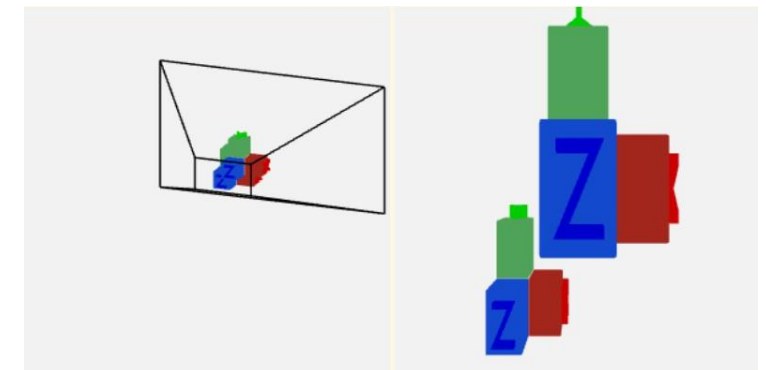
- How do changes of FOV and aspect ratio affect our image?



FOVY = 40; Aspect 4:3



FOVY = 80; Aspect 4:3



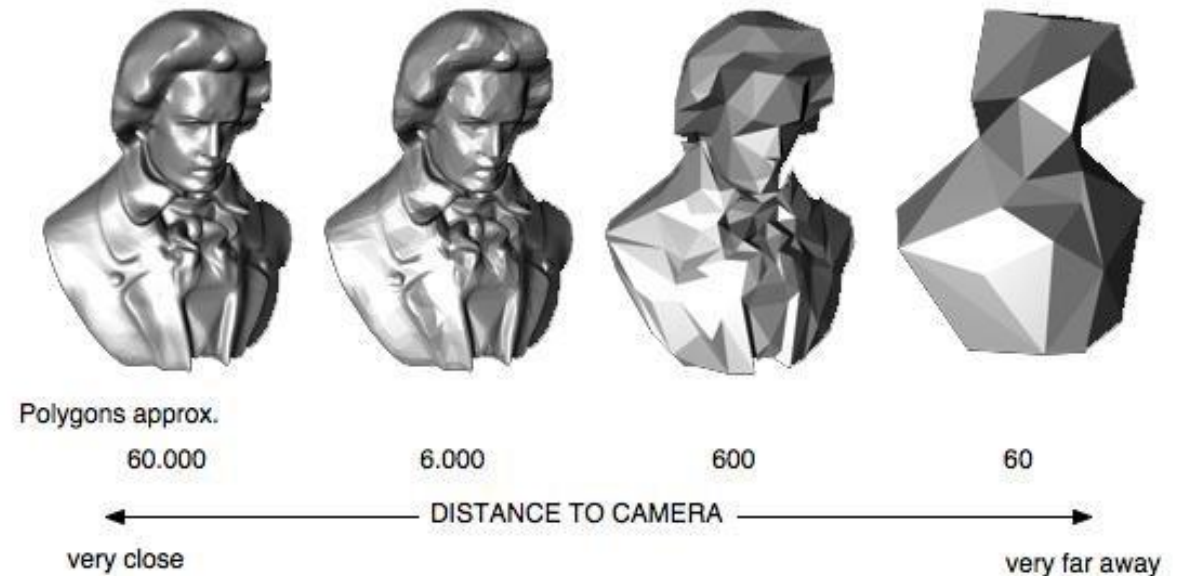
FOVY = 40; Aspect 16:9

Graphics & Rendering

Level of Detail

→ Level of Detail (LOD)

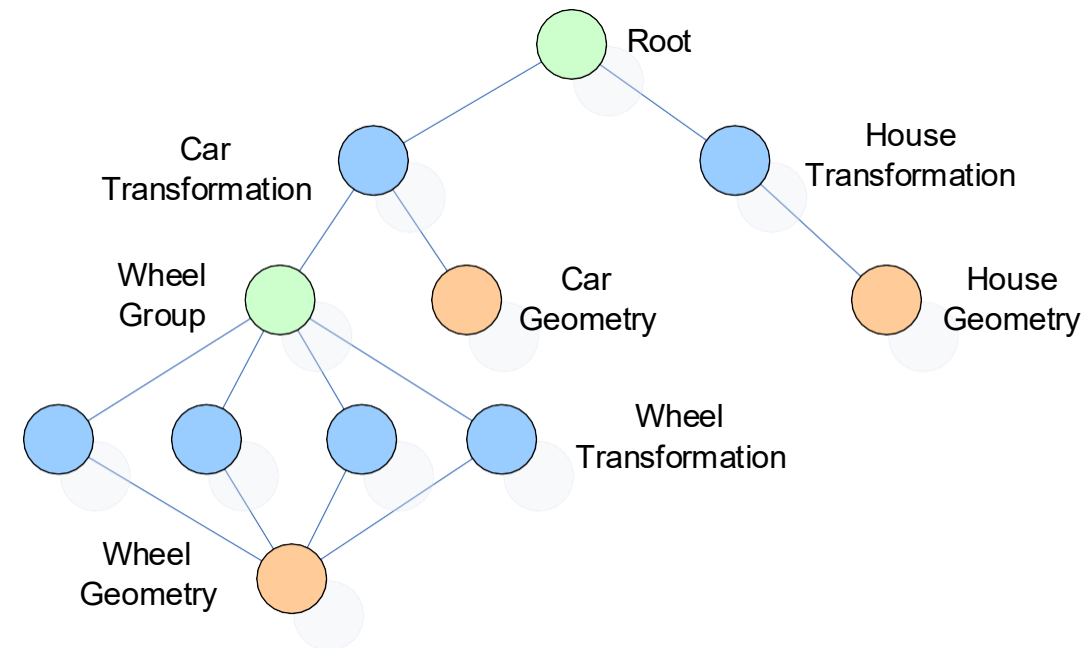
- Used to **increase rendering performance by reducing the amount of drawn triangles**
- **Same 3D object** is reproduced with **different number of triangles**
- Choice of displayed object depending on distance between object and the camera
- Ideally, the different displayed levels of detail are not distinguishable for the user
- Can be pre-processed or calculated dynamically
- Mesh simplification is complex, computationally intense, and error prone
- Variety of optimization algorithms exist



Graphics & Rendering

Scene Graphs

- Used to provide a **higher abstraction layer for graphics programming** compared to low-level APIs (used as well in file formats and game engines)
- Scene graphs are used in 3 **key areas** to support graphics programming
 - Object representation
 - Interactivity
 - Architecture
- Organized as a Directed Acyclic Graph (DAG)
- Follow the concepts of nested transformations
- **Structure**
 - Internal nodes are used for hierarchy building
 - Leaf nodes contain geometry
 - Attributes are typically inherited top – down
 - Parent nodes can have more than a single child node



Graphics & Rendering

Scene Graphs

→ Hierarchy

- Structure lends itself naturally to intuitive spatial organization

→ Culling

- Removing everything from a scene that does not contribute to the final image
- Frustum culling compares object boundaries with viewing frustum
- Occlusion culling checks whether objects are hidden behind others (not often implemented in scene graphs)

→ Bounding volume hierarchy

- Parent nodes bounding volume contains all volumes of child nodes
- If parent is culled away, all of its children are culled away

→ File I/O

- Reading and writing of 3D models from disk
- Internal data structure allows the application to easily manipulate dynamic 3D data

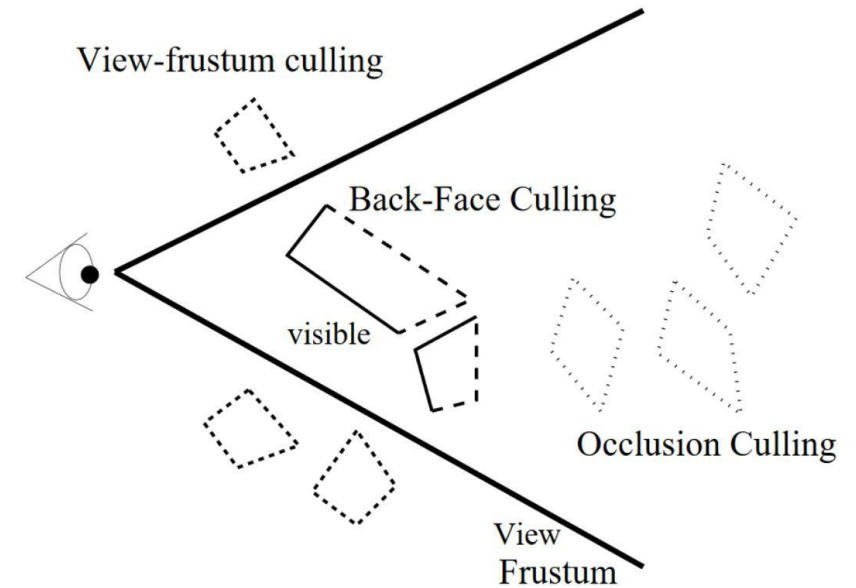
→ Traversal of the scene graph

- Visiting of each node of the graph consists normally of 3 **phases**
- **Update or application**
 - Update traversal allows modifications of the scene graph
 - Updates are applied either directly by the application or with callback functions assigned to nodes within the scene graph
 - Used for animation or input processing
- **Cull**
 - The scene graph tests the bounding volumes of all nodes
 - If leaf node is within the view, a reference to the leaf node geometry is added to a final rendering list
 - List is sorted by opaque versus translucent, and translucent geometry is further sorted by depth
- **Draw or render**
 - List of geometry created during the cull traversal is traversed and low-level graphics API is called to render that geometry

Graphics & Rendering

Culling Techniques

- While clipping will cut off parts during the rendering process, **culling** is used earlier to **remove whole polygons which are not visible**
- Optimization is often required or desired since calculation of color and light can be expensive
- **Only visible parts will contribute to the final image**
- **Most common techniques**
 - Frustum culling
 - Backface culling (provides typically biggest performance increase)
 - Occlusion culling (provides typically lowest performance increase, becomes especially relevant with dense scenes with high number of polygons)
 - Contribution culling (if objects become too small they are simply not rendered anymore)



Graphics & Rendering

Culling Techniques – Frustum Culling

- Is active if object is outside the region of the clipping planes
- Simplified frustum culling test
 - Assuming our view vector is aligned with the z-axis
 - $p_{\text{near}} < z_{\text{view}} < p_{\text{far}}$
 - X- and Y-Axis are centric inside the viewing volume
 - $-w_{\text{view}} < x_{\text{view}} < w_{\text{view}}$
 - $-h_{\text{view}} < y_{\text{view}} < h_{\text{view}}$
- Two simple comparisons are required for each axis
- Still not ideal since test has to be performed for each triangle or primitive
- The naive frustum culling needs $O(n)$ tests where n = number of objects, primitives or triangles
- Typically, use of acceleration structures

Graphics & Rendering

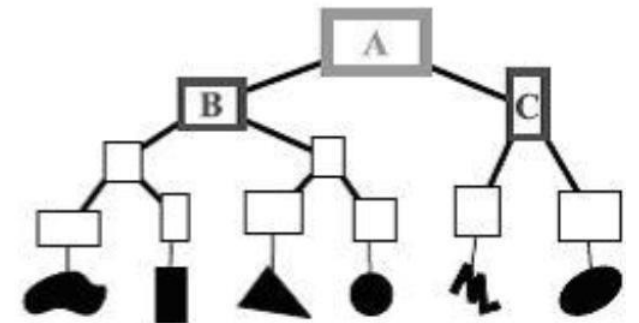
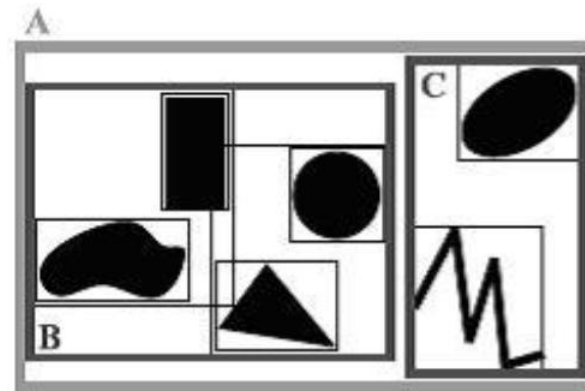
Culling Techniques – Frustum Culling – Optimization

→ Use of **bounding volumes**

- Instead of testing each triangle or primitive a bounding volume encapsulating an object based on primitives is used for testing, if the bounding volume is out of the frustum no further tests are needed
- Bounding volumes are typically axis aligned bounding boxes (AABB)

→ **Bounding volume hierarchy (BVH)**

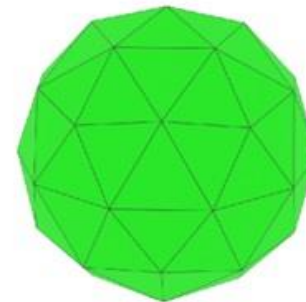
- If hierarchical structure of the scene is available, the parent objects bounding volumes would encapsulate their children's bounding volumes
- If parent bounding volumes is out of the frustum no child volumes would have to be tested



Graphics & Rendering

Culling Techniques – Backface Culling

- The simplest approach of removing triangles for rendering is backface culling
- If the polygons on the back side of objects of an object form a closed surface and we perceive them from the outside, we have no need for drawing the back faces of the polygons
- The polygons on the back side of objects face backwards
- We use the normal vector of the polygon to check for orientation
 - Typically, normal vectors are stored in face mesh structure
 - If not, we can calculate the normal vector as cross product of 2 triangle edges



View from Camera (front)

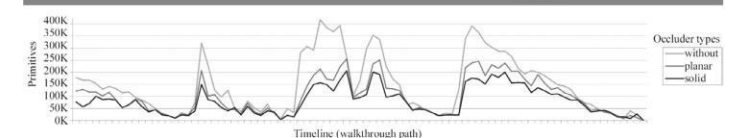
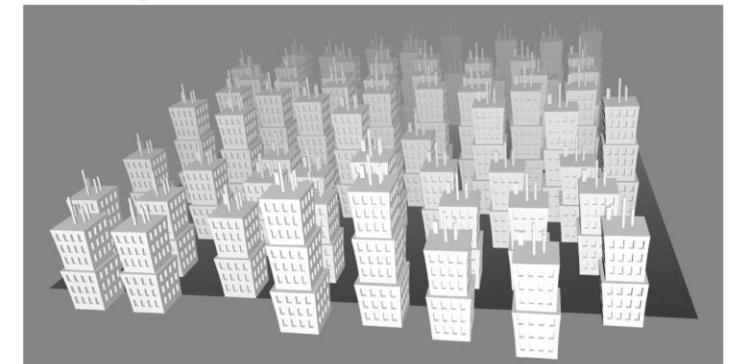
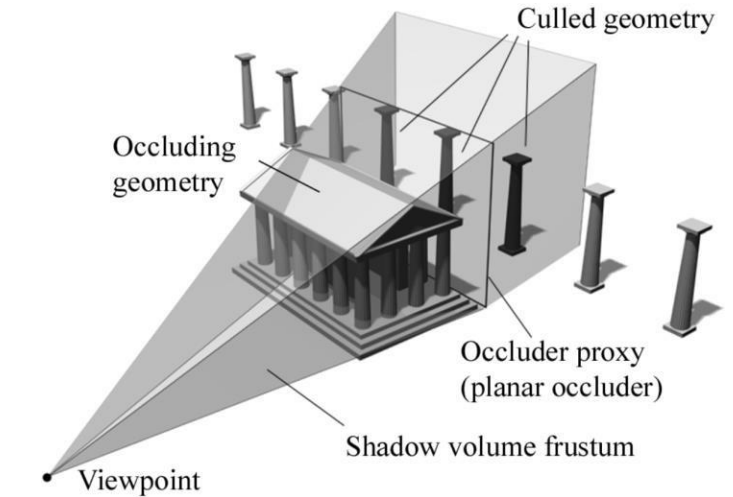


View from Side
(Backface removed)

Graphics & Rendering

Culling Techniques – Occlusion Culling

- **Objects that are completely hidden behind other objects in the scene do not have to be drawn**
- Has to be considered globally, while backface culling and frustum culling can be considered locally
- **Planar Occluders**
 - Uses projection of objects on viewing plane blocking area for depth tests later on
 - Can be implemented by scene geometry or using separate occlusion proxies
- **Shadow Volumes**
 - Creation of a shadow volume between the viewpoint and the far clipping plane
 - More on that in texture and buffer part of the lecture



- The **intensity** of the light received from a surface in terms of color (based on the chosen color model) depends on
 - The light sources in the environment
 - The structure and other properties of the material hit by the light
 - The description on how the light is reflected (illumination or reflectance model)
 - The description on how the surface is shaded (shading model)
- **Light sources**
 - Light sources are used to simulate physical lights in the scene
 - Depending on the chosen model this can be unrealistic and computationally cheap or realistic and computationally expensive
 - They have a given intensity and colour
 - Their position and type as well as, depending on the type of light source, orientation is important

Graphics & Rendering

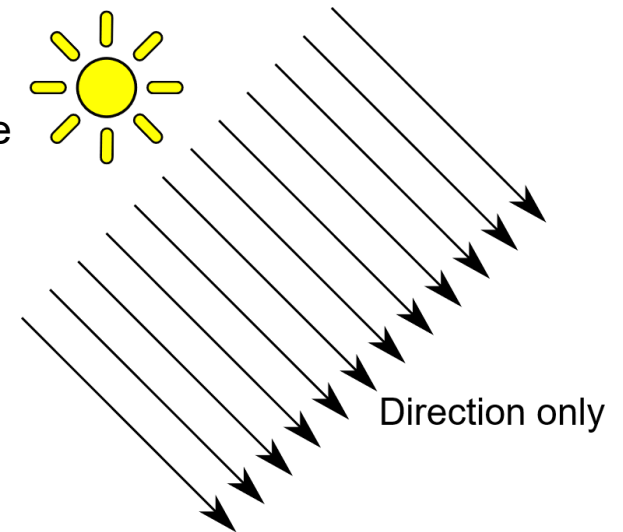
Light Sources

→ Ambient Light (indirect)

- A light source which is not directed
- Can be considered as environmental light
- Light is evenly distributed in all directions
- Creates low contrast images

→ Directional Light (direct)

- Light is emitted from a far distance, so that the rays are considered parallel
- The intensity of the light is independent of the distance between objects and the light source
- The range of a directional light is typically unlimited. (e.g. sun)

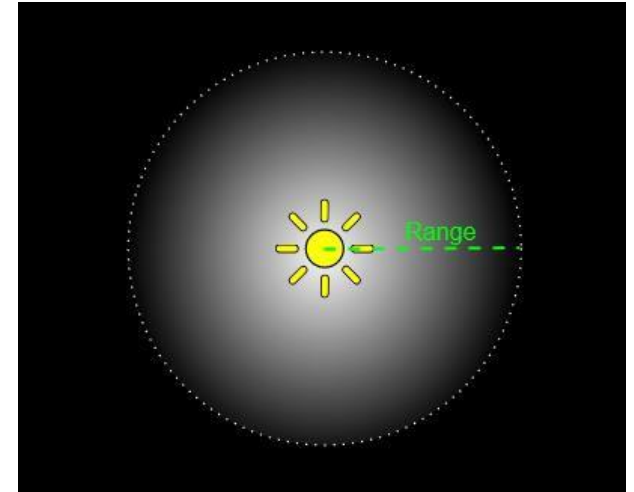


Graphics & Rendering

Light Sources

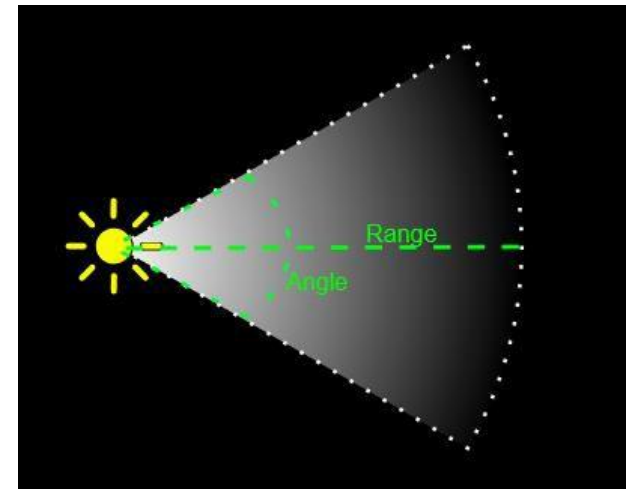
→ Point Light (direct)

- Light source with a single point of origin illuminating equally in all directions
- Depending of the distance from the illuminated objects to the light source the intensity of the light is determined
- Range is typically limited (e.g. light bulb)



→ Spot Light (direct)

- Is a point light emitting light with a given opening angle
- A direction for the central light ray is required
- The range is as with a regular point light limited
- The intensity depends as well on the distance between light source and objects (e.g. flashlight)



Graphics & Rendering

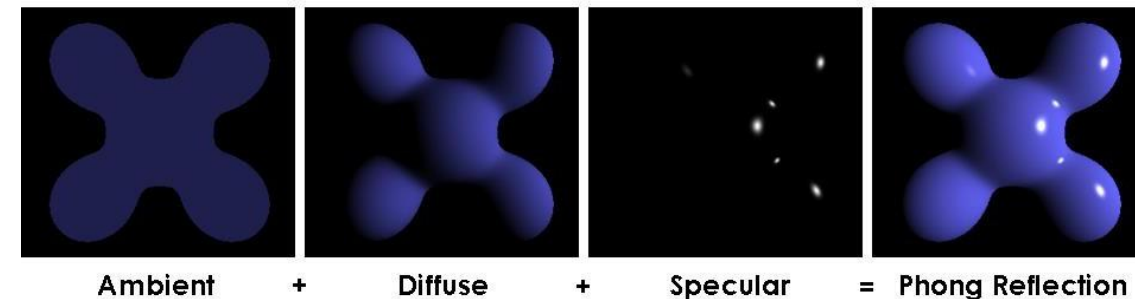
Illumination and Reflectance

→ Illumination

- Process of calculating how light interacts with surfaces in a scene to simulate realistic lighting for visual fidelity
- **Key Components**
 - Light sources (point, directional, ambient)
 - Surface properties (color, texture, material)

→ Reflectance

- Proportion of light striking a surface which is reflected off it
- Ambient
 - illuminates all surfaces equally (simulates indirect light)
- Diffuse
 - matte appearance; direction-dependent
- Specular
 - glossy appearance, creates shiny highlights; view-dependent
- Phong
 - combines ambient, diffuse, and specular components for realistic shading

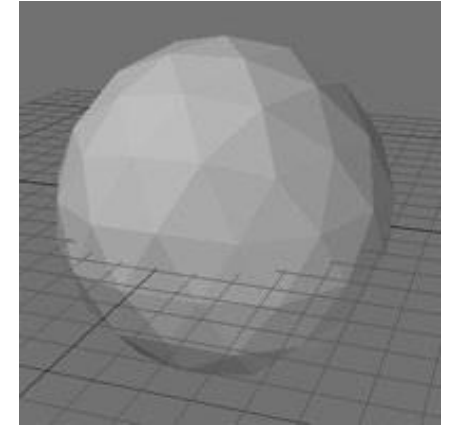


Graphics & Rendering

Shading

→ Flat or Constant Shading

- The most simple and fast shading model is the flat shading
- A whole triangle is assigned a single color value
- The color of the triangle can be determined in multiple ways
 - The color of the first vertex with its normal can be used
 - The color of the centroid can be used with help of the face normal
 - The color of the averaged vertices of the triangle can be used
- Between the individual triangles the edges become visible, leading to clearly visible distinct surfaces (Mach-band effect) exaggerating contrast
- Transparency, shadows and highlights are normally ignored with flat shading
- Can find use in applications, where a high number of triangles needs to be rendered simultaneously and the display quality is not as important (e.g., industrial CAD files)
- Can also be used for artistic purposes to generate a retro look (e.g., digital games)
 - “Cartoon-like appearance”



Graphics & Rendering

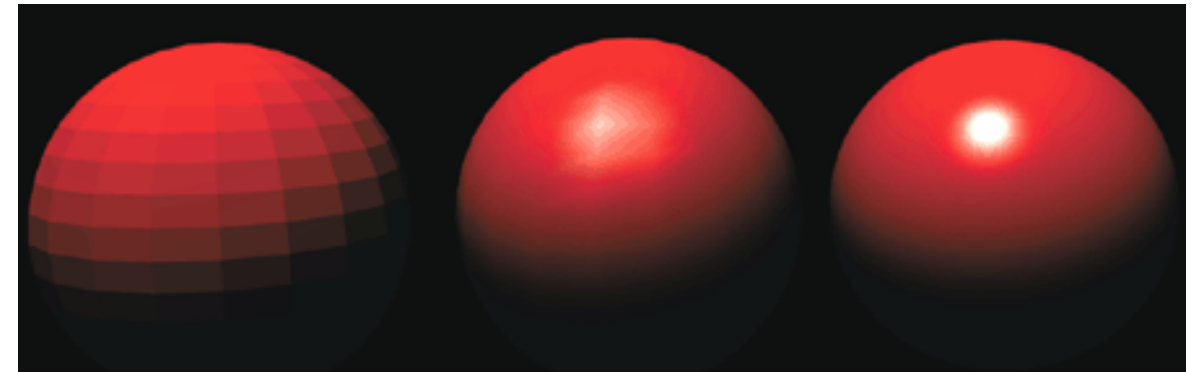
Shading

→ Gouraud Shading

- Calculates the individual colors of the vertices
- The color of the triangle is determined by interpolation of the vertex colors

→ Phong Shading

- Similar idea than Gouraud shading, but instead of interpolation of the color intensities, normal vectors are interpolated
- More realistic than Gouraud shading
- Requires more computation



Flat

Gouraud

Phong

Graphics & Rendering

Textures

- With shading, colorful images of geometries can be created but often more detail is desired
- A **texture** in general is used to **add surface details to a 3D model**
- That could be hatching, patterns, whole images but also other attributes like wrinkling of the surface, terrain, reflections of the environment, transparencies
- **2D images are used and mapped onto a 3D geometrical object**
- Think of it as shrink wrap or gift-wrapping paper
- Parts of the texture could be mapped individually only on parts of the object



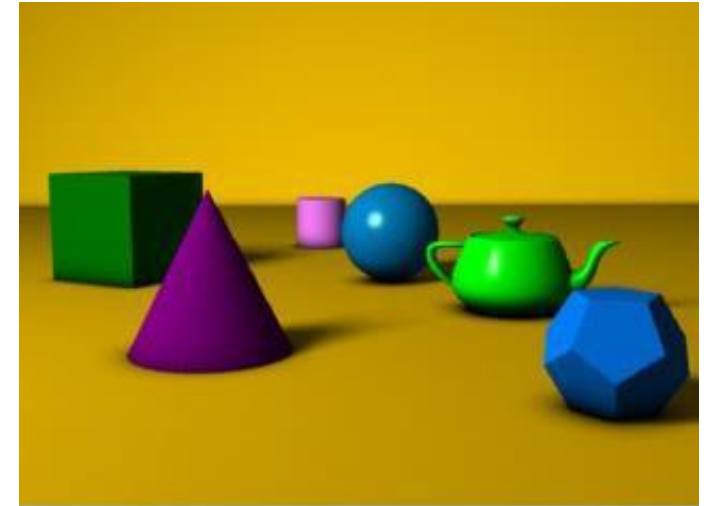
Graphics & Rendering

Z-Buffer or Depth Buffer

→ Filling the z-buffer

→ **Algorithm**

- **For each polygon in the Frustum**
 - Determine the pixels to be drawn
 - **For each pixel to be drawn**
 - Compute the depth value z (distance to the camera) at the x,y position
 - If $z(x,y) < \text{current value}(x,y)$ then $\text{current value}(x,y) := z(x,y)$
- **Render image**



Game Engine Concepts



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

FH Oberösterreich

University of Applied Sciences Upper Austria

Digital Media Department

Media Technology & Design

Digital Arts

Interactive Media



Dr. Andreas Riegler

Professor of Interactive Systems

Digital Media Department

E Andreas.Riegler@fh-hagenberg.at